Project Title: Intelligent Pathfinding using A* Search Algorithm

1. Introduction

Pathfinding is a fundamental problem in Artificial Intelligence, critical for robotics, video games, and GPS navigation systems. It involves finding the optimal route between two points while avoiding obstacles. This project implements the *A (A-Star) Search Algorithm**, which is widely regarded as one of the most effective pathfinding techniques due to its completeness, optimality, and efficiency

2. Objective

The main objectives of this project are:

- To simulate a grid-based environment containing open spaces and obstacles
- To implement the A* algorithm to calculate the shortest path from a "Start" node to a "Goal" node
- To visualize the search process and the resulting path graphically

3. Methodology

The solution is built using Python. The core logic relies on the A Algorithm*, which evaluates nodes based on a cost function f(n):

- f(n) = g(n) + h(n)
- **g(n):** The actual cost from the start node to the current node n.
- **h(n):** The estimated cost (heuristic) from node n to the goal.

Heuristic Used:

We utilized the Manhattan Distance heuristic, which is ideal for grid-based maps where movement is restricted to four directions (up, down, left, right). It calculates the absolute sum of the coordinate differences: |x_1 - x_2| + [cite_start]|y_1 - y_2|

Data Structures:

- **Priority Queue (heapq):** Used to efficiently retrieve the node with the lowest f(n) score for exploration
- **Dictionaries:** Used to track the "parent" of each node (to reconstruct the path) and the current cost scores (g and f).

4. Code and Implementation Details

The implementation consists of three main components:

1. **Grid Class:** Represents the environment. It checks if moves are valid (within bounds and not hitting obstacles)
2. **AStar Class:** Contains the search logic. It expands nodes, calculates costs, and updates the priority queue. If the goal is reached, it backtracks using the came_from dictionary to reconstruct the path
3. **Visualization:** using the matplotlib library, the grid is rendered visually. Obstacles are plotted as gray blocks, the start and end points as markers, and the calculated path is highlighted with 'X' marks

(Note: The full main.py source code provided in the zip file contains the complete implementation.)

5. Results and Observations

The algorithm successfully identified the shortest path between the defined start coordinates (1, 1) and goal coordinates (8, 8).

- **Obstacle Avoidance:** The agent successfully navigated around the manually placed "wall" of obstacles.
- **Optimality:** By using the A* heuristic, the system explored fewer nodes than a standard Breadth-First Search (BFS) would have, demonstrating efficiency.
- **Visualization:** The generated plot clearly indicates the grid layout and the optimal route taken by the agent.

6. Conclusion

This project demonstrates the practical application of heuristic search algorithms in AI. The A* algorithm proved to be robust and efficient for solving the maze problem. This simulation serves as a foundation that can be expanded into more complex domains, such as autonomous robot navigation or dynamic route planning in traffic systems